



INSTITUTO TECNOLÓGICO DE COSTA RICA
ESCUELA DE INGENIERIA EN COMPUTACIÓN

CENTRO ACADEMICO DE LIMÓN

CODIGO : IC1803

GRUPO 61

PROYECTO BUSCAMINAS

REALIZADO POR:

Kerry Hernández Alcázar 2026014741

Contenido

1. Configuración inicial de la partida.....	3
1.1 Selección de dificultad:	3
1.2 Selección de modo de juego:.....	3
1.3 Iniciar partida:	3
2. Mecánicas del tablero	3
2.1 Clic Izquierdo:	3
2.2 Clic Derecho:	4
3. Gestión de partida	4
4. Condición de victoria y ranking.....	4
5. Descripción del problema	5
6. Diseño del programa	5
Decisiones de Desarrollo.....	5
Algoritmos usados	5
7. Librerías usadas.....	6
8. Análisis de resultados.....	6
Objetivos Alcanzados	6
Objetivos no alcanzados.....	6
9. Conclusión	7

1. Configuración inicial de la partida

1.1 Selección de dificultad:

Utilice el menú para desplegar la dificultad de la partida:

Fácil: Matriz de 8 x 8 con 10 minas

Medio: Matriz de 12 x 12 con 25 minas

Difícil: Matriz de 16 x 16 con 50 minas ocultas

1.2 Selección de modo de juego:

El juego provee 2 modalidades de juego:

Modo Normal: Modo de juego casual, sin límite de tiempo

Modo Contrarreloj: Modo de juego enfocado en quienes quieran un desafío, con una cuenta regresiva según el nivel de dificultad (Fácil: 3 min, Medio: 8 min, Difícil: 15 min). Si el contador llega a cero el jugador pierde

1.3 Iniciar partida:

Botón de juego “Iniciar Juego” que crea el tablero de juego según las especificaciones del usuario

2. Mecánicas del tablero

El juego se controla de manera interactiva utilizando los clics del mouse sobre las celdas grises de la cuadrícula

2.1 Clic Izquierdo:

- Revela el contenido secreto de una mina

- Si la casilla contiene una mina el juego se termina y muestra el mensaje de que el juego ha terminado
- Si la celda casilla tiene una o varias minas a la par se mostrara un numero del 1 al 8 según cuantas minas posea alrededor
- Si la casilla esta vacía (0) se usa la función de recursividad para revelar las casillas en forma de cascada, revelando las casillas vacías hasta toparse con alguna que posea una mina alrededor

2.2 Clic Derecho:

- Coloca una bandera sobre la casilla que el jugador piense que pueda haber una mina
- Mientras la casilla este marcada quedara bloqueada para prevenir algún clic accidental
- Al hacer clic sobre una casilla ya marcada la bandera será removida devolviendo la casilla a su estado original

3. Gestión de partida

En la parte de abajo del tablero están presentes 2 botones de acción rápida

Reiniciar: Limpia el tablero actual, vuelve a distribuir las minas y vuelve a iniciar el cronometro dese 0

Abandonar: Cierra la ventana de juego actual y devuelve al jugador al menú principal

4. Condición de victoria y ranking

Victoria: El juego se gana al revelar todas las casillas que no posean una mina en su contenido. No es obligatorio marcar todas las minas para ganar, basta con solo dejar las minas ocultas

Ranking: Al tener una partida en condición de victoria el código calculará el tiempo que duro la partida más sus estadísticas. El código guardara los rankings del menor a mayor tiempo y guardara los datos en el archivo ranking.txt. Los mejores 10 partidas se guardaran de manera persistente entre sesiones y pueden ser consultados desde el menú de inicio con el botón “Ver Rankings

5. Descripción del problema

El proyecto consiste en el desarrollo de un juego de **Buscaminas** basado en el uso de lógica de matrices, uso de recursividad (efecto cascada), manejo de archivos (rankinga.txt) y uso de condicionales para determinar parámetros como la condición de victoria o la creación del tablero $n \times n$ dividiéndose en 2 áreas:

Nueva partida: Crea una matriz $n \times n$ según la dificultad seleccionada

Ver rankings: Muestra los mejores 10 partidas según el tiempo con el que se realizaron

6. Diseño del programa

Decisiones de Desarrollo

1. Uso de 2 matrices: Para garantizar las reglas del juego, se necesita el uso de 2 matrices. La matriz lógica que es la que guarda los valores reales de cada casilla (minas, número de proximidad), mientras que la matriz visible únicamente maneja una matriz en estado "oculto" además de ser la que maneja los parámetros de casillas ocultas, reveladas o marcadas con banderas. Esto previene que el usuario pueda acceder a la solución desde la interfaz
2. Uso de memoria global: Se optó por centralizar el estado del juego mediante variables globales controladas desde funciones específicas. Esto facilita la comunicación interactiva y facilita el uso de componentes dinámicos de la interfaz gráfica sin comprometer el rendimiento
3. Persistencia Basada en Texto: Se decidió utilizar un archivo de texto para guardar las mejores 10 partidas, esto con la finalidad de poder acceder a dicha información de manera sencilla sin depender de que el código tenga que guardar toda la información

Algoritmos usados

1. Cálculo de Vecinos: Después de que el tablero es creado ya con las minas integradas aleatoriamente (random.randint) el código a partir de una casilla ya descubierta escanea sus 8 coordenadas vecinas mediante desplazamientos (-1,0,1), validando límites perimetrales para evitar desbordamientos de índice. Cada mina cercana incrementa en un el valor de la casilla de origen.
2. Efecto cascada: Cuando el jugador descubre una celda de valor 0 el código desencadena una función recursiva de apertura. Si la celda actual es válida y está oculta, se desvela de inmediato, si su valor es cero, vuelve a llamar a la

misma función para sus 8 celdas vecinas, la recursividad se detiene hasta que se encuentra una casilla que sea mayor a cero y cuando ya no hayan más casillas que desvelar

7. Librerías usadas

1. tkinter: La biblioteca principal para la creación de la interfaz gráfica. Provee lo contenedores de ventanas (Tk, Toplevel). Los marcos organizacionales (Frame), las etiquetas informativas (Label), las entradas de texto (Entry) y las matrices de botones interactivos (Button)
2. Tkinter.messagebox: Subbiblioteca encargado de crear ventanas emergentes de diálogo, se usa para suspender el flujo principal y notificar estados determinados al jugador, alertas como las de victoria o derrota
3. random: Utilizada para la creación y distribución de las minas en todo el tablero
4. time: Biblioteca utilizada para la gestión del tiempo en el programa. Utiliza time.time() para capturar las marcas de tiempo cuando una partida finaliza, esto con la finalidad de poder guardar los mejores récords de manera mas sencilla

8. Análisis de resultados

Objetivos Alcanzados

1. Se logró que el juego pudiese tener las 3 dificultades, cada una con sus especificaciones del tablero, minas y el tiempo para completar
2. Se logro implementar recursividad en el efecto cascada para revelar casillas vacías
3. Se logro que el código pueda guardar y comparar los mejores tiempos en un archivo de texto
4. Se logró implementar satisfactoriamente tanto el cronometro ascendente del modo normal así como el temporizador de cuenta regresiva del modo contrarreloj

Objetivos no alcanzados

1. Lograr que al reiniciar una partida el juego se mantenga en una sola ventana sin necesidad de abrir otra ventana

9. Conclusión

Este proyecto en mi punto de vista resultó ser un verdadero reto, no por la lógica de mantener constancia a la hora de usar 2 matrices o por la lógica de usar recursividad para el efecto cascada, sino más bien por presentarse como un reto al conocimiento de un estudiante de primer ingreso, no solo refleja el esfuerzo y dedicación que se debe poner en futuros proyectos cuando se de la carrera por terminada, también refleja el compromiso que se debe tener con esta clase de proyectos, el manejo del tiempo para investigar y codificar funciones lógicas que ayuden a la solución del problema, comprender la lógica detrás del juego y trabajar funciones en base a ello. Sin duda la realización de este proyecto abre una ventana hacia cada estudiante, hacia su dedicación, su determinación y su fuerza mental para la finalización de dicho proyecto.